TECHNICAL HANDOFF · INTERNAL

Nolan

Documentary Footage Studio — Architecture & Developer Guide

Repository: github.com/mavestone/nolan-studio

Platform: macOS desktop app (local server + browser UI)

Stack: Python 3.11 · FastAPI · SQLite · faster-whisper · PySceneDetect · Claude / Groq / Gemini

Codebase: ~13,700 LOC · 7 Python modules · single-file SPA frontend

Document date: 13 June 2026

> Nolan ingests raw documentary footage, transcribes it locally, detects & visually tags scenes, and lets an editor **search by what's said and what's on screen** — from the browser or a Telegram bot — then exports editorial cuts as FCPXML.

— Contents

1 System Overview & Tech Stack

Nolan is a **single-user macOS desktop application**. There is no cloud component — everything runs locally. A native Swift launcher (`Nolan.app`) starts a Python FastAPI server on `localhost:8765` and opens the default browser to it. The "app" is therefore a local web app; the browser is the UI, the Python process is the backend, and an SQLite file is the database.

1.1 What it does

- **Ingest** — scans folders of `.mp4` / `.mov` raw camera footage.
- **Transcribe** — local Whisper (faster-whisper), fully offline, no API needed.
- **Scene-tag** — samples each clip, classifies shot size / angle, indoor-outdoor, dialogue density, and (via Claude vision) the concrete objects on screen.
- **Search** — by transcript text, by shot attributes, or by "vibe" (LLM-expanded semantic search). Available in-browser and via a Telegram bot.
- **Story analysis** — an LLM "story bible" (characters, themes, locations, arc) over the whole project.
- **Export** — AI-selected narrative cuts rendered to **FCPXML** for DaVinci Resolve / FCP.

1.2 Dependencies (pinned — `requirements.txt`)

LAYER	PACKAGE	VERSION	ROLE
Web	`fastapi / uvicorn`	`0.115.12 / 0.34.2`	HTTP server + ASGI
Web	`starlette`	`>=0.46,<0.47`	Pinned — 1.0 broke FastAPI compat
DB	`aiosqlite`	`0.22.1`	Async SQLite
ASR	`faster-whisper / av`	`1.2.1 / 17.0.1`	Local transcription (bundled decoder)
Vision	`scenedetect`	`>=0.6.5`	Cut detection (fallback path only)
Vision	`opencv-python-headless`	`>=4.10,<5`	Heuristic shot classification
AI	`anthropic / groq / google-genai`	`0.97 / 1.2 / 1.74`	LLM + vision backends
Bot	`python-telegram-bot[job-queue]`	`22.7`	Telegram interface (polling)
Config	`python-dotenv`	`1.1.0`	.env loading

> **EXTERNAL BINARY** **ffmpeg & ffprobe** are required at runtime for thumbnail extraction and duration probing. They are **not** Python packages — installed via Homebrew by `install.sh`. See §14 for the PATH trap this causes in `.app` launches.

1.3 Entry point

```
# main.py — bottom of file
if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=8765)
```

Single worker, single process. All concurrency is async (asyncio) plus a thread pool for CPU/IO-bound work (transcription, ffmpeg). There is intentionally no multi-worker setup — the app is single-user and holds in-memory job state.

2 Architecture

```
Nolan.app (Swift launcher)
  └─▶ spawns: python main.py  (cwd = repo root, PATH patched)
      ├─ FastAPI / uvicorn  :8765
      ├─ SQLite (footage.db, WAL mode)
      ├─ ThreadPoolExecutor  (whisper, ffmpeg, scene detect)
      └─ asyncio task: Telegram bot (long-poll)
  └─▶ opens browser → static/index.html (SPA) ⇄ /api/* (polling)
```

2.1 The three concurrency lanes

LANE	WHAT RUNS THERE	WHY
asyncio event loop	All HTTP handlers, DB calls (aiosqlite), Telegram bot	IO-bound, cheap to interleave
ThreadPoolExecutor	Whisper transcription, ffmpeg subprocess, OpenCV, scene detect	CPU/blocking — released GIL means real parallelism
FastAPI BackgroundTasks	Batch process orchestration, scene backfill runner	Fire-and-forget after HTTP response

2.2 In-memory job state

There is a module-level dict `job_progress: dict[int|str, dict]` in `main.py` that is the single source of truth for live progress. The frontend polls `/api/jobs` every ~2.2s and reads it. Keys are either a **file_id** (per-clip progress) or a **sentinel string**:

KEY	MEANING
`<int>`	Per-file progress: `{status, progress, stage, filename}`
`__scan__`	Folder scan status
`__scene_backfill__`	Bulk scene (re)detection: `{running, done, total, workers, ai}`
`__reclassify__`	OpenCV reclassification pass
`reanalyze_<pid>`	Project-wide AI vision re-run
`__project_analysis__`	Story-bible generation

> **CONSEQUENCE** Because job state is in-memory, a server restart loses live progress (the work itself is checkpointed in the DB). `reset_stuck_states()` runs at boot to revert any clip left mid-flight back to a clean status.

2.3 Activity log ring buffer

A custom `logging.Handler` (`_ActivityHandler`) mirrors every `nolan` log record into a 600-entry `deque` . `GET /api/activity?since=<seq>` serves incremental lines to the in-app console drawer. This is how the UI shows "what it's doing" live.

3 Module Reference

Seven Python modules + a three-file frontend. No package structure — flat repo, imported by name.

FILE	LOC	OWNS
`main.py`	2457	FastAPI app, all routes, job orchestration, startup maintenance, settings
`database.py`	902	Schema + migrations, all SQL, the `_db()` WAL connection ctx-manager
`telegram_bot.py`	2051	Bot handlers, vibe/search/cut over chat, capture-mode state machine
`analyzer.py`	948	LLM abstraction (4 backends + fallback), story bible, cut selection, chat
`scene_detector.py`	634	Sampling + cut detection, OpenCV classifier, Claude vision, ffmpeg calls
`fcpxml.py`	392	FCPXML generation, SMPTE timecode read via ffprobe
`transcriber.py`	94	faster-whisper wrapper, folder scan, audio-stream check
`static/app.js`	2734	SPA: state, rendering, polling, all UI logic
`static/style.css`	3017	Styling (dark theme)
`static/index.html`	455	DOM skeleton, loads `app.js?v=N` (bump N to bust cache)

CACHE BUSTING The frontend is unbundled. After editing `app.js` or `style.css`, bump the `?v=NN` query string in `index.html` or the browser serves stale JS.

4 Data Model

SQLite at `footage.db` (repo root), opened in WAL mode with `busy_timeout=10000` and `foreign_keys=ON` via the shared `_db()` async context manager. Nine tables.

4.1 Tables

TABLE	KEY COLUMNS	NOTES
`projects`	id, name, created_at	Top of the tree
`project_folders`	id, project_id→, path, last_scanned, files_found	Source dirs; `UNIQUE(project_id,path)`
`files`	id, project_id→, path, filename, duration_seconds, status, +12 denorm cols	One row per clip. See 4.2
`segments`	id, file_id→, start_time, end_time, text	Transcript lines
`scenes`	id, file_id→, scene_num, start/end, thumbnail_path, +9 attr cols	Sampled "scenes". See 4.3
`analysis`	file_id→, story_value, summary, highlights, themes, characters, raw_json	Per-clip LLM analysis
`project_analysis`	project_id→, raw_json	Whole-project "story bible"
`bible_batch_checkpoints`	project_id→, batch_num, total_batches, summary	Resumable bible generation
`chat_messages`	id, project_id→, role, content, pinned	Per-project chat history

4.2 `files` — denormalized scene summary columns

Scene attributes are rolled up onto the file row for fast list sort/filter (`update_file_scene_summary()`):

`poster_path · primary_shot_type · primary_shot_size · primary_roll_type · primary_setting · primary_location · scene_tags(json)`

status ∈ `pending · queued · transcribing · transcribed · analyzing · done · silent · error` . `silent` = no audio stream (b-roll); still gets scenes.

4.3 `scenes` — the searchable unit

COLUMN	VALUES	SET BY
`thumbnail_path`	static/thumbnails/<fid>/NNNN.jpg	ffmpeg seek
`shot_size`	extreme_close_up · close_up · medium · full · wide · extreme_wide · aerial	OpenCV, refined by AI
`shot_angle`	eye_level · low · high · dutch · over_shoulder · pov	OpenCV / AI
`roll_type`	no_dialogue · dialogue · heavy_dialogue (legacy: a_roll/b_roll)	transcript word-density
`setting`	indoor · outdoor	**AI only** (OpenCV too unreliable)
`location`	free text ("desert dune", "kitchen")	AI vision
`description`	one-sentence composition	AI vision
`visual_content`	comma list: "shoe, sand, hand, camel, water"	AI vision — **powers object search**
`ai_classified`	0 / 1	1 once Claude vision ran

> **"FULLY PROCESSED"** A clip is considered done only when it has **at least one scene with a thumbnail AND at least one scene with visual_content** (`has_scene_thumbnails()` + `has_scene_ai()`). The Pending tab = anything short of that.

5 The Processing Pipeline

Triggered by `POST /api/projects/{id}/process`. The handler partitions the project's clips into work queues, then returns immediately; work runs in background tasks.

```
scan (folder → file rows)
  ▼
1. transcribe  clips not yet done  (faster-whisper, threadpool)
  ▼ per clip
2. scene detect  (sample → thumbnail → classify)
  ▼ same pass when AI on
3. AI vision  (Claude Haiku per scene thumbnail → visual_content)
  ▼ on demand
4. story bible  (batched LLM over all transcripts)
```

5.1 Partitioning logic (the important part)

On Process, three buckets are computed:

1. **to_transcribe** — status not in (done, transcribing, queued, silent).
2. **to_scene_only** — done/silent clips that are **not fully processed** (missing thumbnail OR missing AI), **and on disk**. Missing-on-disk clips are counted and skipped (need relink).
3. Transcribed clips flow into batch analysis automatically.

```python
# Selection — checks usable data, NOT mere row existence
has_thumb = await has_scene_thumbnails(f["id"])
has_ai    = await has_scene_ai(f["id"])
if has_thumb and has_ai:        # fully processed → skip
    continue
real = _resolve_actual_path(f["path"])
if not os.path.exists(real):     # missing on disk → skip + count
    skipped_missing += 1; continue
to_scene_only.append(f)
```

> **HISTORICAL BUG — READ IT** The original check was `has_scenes()` (row exists). But a failed thumbnail extraction (drive unplugged) left empty scene rows, so `has_scenes()` was true and Process skipped them forever. Always gate on **usable data**, not row presence.

5.2 The scene backfill runner

Clips are processed concurrently via a dedicated `ThreadPoolExecutor` bounded by an `asyncio.Semaphore`. Worker count defaults to `CPU−1` (clamped 2–8), reduced to 3 when AI is on. Each clip: resolve path → load transcript → `detect_scenes(use_ai=...)` → `save_scenes` → `update_file_scene_summary` → set poster. Respects a global `_stop_requested` flag.

6 Scene Detection — Deep Dive

This subsystem went through the most iteration; the design reflects hard-won performance lessons on real 4K HEVC footage stored on a slow external drive.

6.1 Two modes

MODE	FUNCTION	WHEN
Sampling (default)	`detect_scenes_sampled()`	Raw footage — single continuous takes, no internal cuts
Cut detection (fallback)	`PySceneDetect AdaptiveDetector`	`scene_detect_mode:"cuts"` — edited clips

6.2 Why sampling, not cut detection

Raw camera clips have no cuts. Full-frame cut detection must **decode every frame** to confirm that — ~35s for a 44s 4K HEVC clip — purely wasted. Sampling instead **seeks** to a handful of timestamps (~0.8s each via keyframe seek) and grabs a thumbnail. **8–10× faster**; a 44s clip → 4.4s.

APPROACH (TESTED ON REAL 4K HEVC)	TIME/CLIP	VERDICT
PySceneDetect full decode	~35s	Accurate but wasteful for raw
frame_skip on PySceneDetect	~35s	No gain — HEVC must decode ref frames
ffmpeg keyframe-only scene filter	~6s	Misses real cuts (68-scene clip → 0-2)
Time-interval seek sampling	**~4s**	**Chosen** — right model for raw

6.3 Content-adaptive density (search-optimized)

Sampling density follows the transcript, because that's where visual variety is:

- **B-roll** (no dialogue) → dense (every `scene_broll_seconds`, default 6s) — objects/locations live here.
- **A-roll** (dialogue) → sparse (every `scene_aroll_seconds`, default 15s) — static talking head; transcript carries it.
- Capped at `scene_max_samples` (20). When the cap binds on a long clip, samples are evenly thinned across the **whole** clip (not front-loaded), preserving coverage.

6.4 Per-sample classification stack

```
thumbnail.jpg
  ▼ _classify_shot_opencv()  (Haar faces + HSV colour → shot_size, angle, tags)
  ▼ if use_ai & Claude available
  classify_shot_with_claude()  (Haiku vision → setting, location, description, visual_content)
  ▼
roll_type set separately from transcript word-density (_dialogue_level)
```

> **INDOOR/OUTDOOR** The OpenCV colour heuristic is deliberately **not** trusted for setting — it's left null and only the AI vision pass writes it. Descriptive tags (desert/night/nature) still come from OpenCV.

6.5 AI vision call (cost-relevant)

One 320px JPEG thumbnail per scene → **Claude Haiku via the local Claude CLI (OAuth)**, not the metered API. Returns strict JSON. So vision cost is the user's Claude subscription, not per-token API spend.

7 The AI Layer (`analyzer.py`)

All LLM access goes through a small set of `_try_*` functions with a defined fallback order. Each returns text or raises; callers try backends in sequence until one succeeds.

7.1 Backends & models

BACKEND	_TRY_*	AUTH	USED FOR
Anthropic API	`_try_anthropic_api`	ANTHROPIC_API_KEY	Primary when key set
Claude CLI	`_try_claude_cli`	Desktop OAuth (no key)	Vision + fallback
Groq	`_try_groq`	GROQ_API_KEY	Fast text fallback
Gemini	`_try_gemini`	GEMINI_API_KEY	Optional

7.2 Model aliases

```
CLAUDE_MODEL        = "claude-haiku-4-5"     # default — fast/cheap
CLAUDE_SONNET_MODEL = "claude-sonnet-4-5"    # deeper reasoning
CLAUDE_OPUS_MODEL   = "claude-opus-4-7"      # top-tier
GROQ_MODEL          = "llama-3.3-70b-versatile"
```

User-facing names map `haiku|sonnet|opus|groq` → these IDs. The Telegram bot exposes `/model` to switch; the web UI has a model selector for analysis.

7.3 Key flows

FUNCTION	DOES
`analyze_project()`	Batches transcripts, generates per-clip + project "story bible"; checkpointed per batch so it resumes.
`select_narrative_clips()`	Given a theme + duration, picks clips/in-out points for a cut. Returns structured JSON.
`chat_about_project()`	Conversational Q&A grounded in transcripts (web chat).
`chat_transcripts_only()`	Lean Telegram path: LLM query-expansion → wide transcript search → editorial answer + cited clips.

VIBE SEARCH Two-stage: stage 1 expands a "vibe" into spoken-language terms people actually say; stage 2 reads matched transcripts and selects by feel. Lives in `telegram_bot._vibe_search`.

8 HTTP API Reference

All under `/api`. Grouped by concern. Verbs as defined in `main.py`.

PROJECTS & FOLDERS

`POST /api/projects`	create	`GET /api/projects`	list
`DELETE /api/projects/{id}`	delete (cascades)	`GET /api/pick-folder`	native folder picker (osascript)
`POST /api/projects/{id}/folders`	add folder	`.../folders/{fid}/scan`	scan for clips

PROCESSING & JOBS

`POST /api/projects/{id}/process`	run pipeline	`POST /api/stop`	cooperative stop
`GET /api/jobs`	all job_progress	`GET /api/activity?since=`	console log stream
`GET /api/scan/status`	scan progress	`.../reanalyze-scenes`	project-wide AI vision

FILES, SCENES, SEARCH

`GET /api/projects/{id}/files`	clip list (+has_thumbnail/has_ai_detect)	`GET /api/files/{id}`	detail
`GET /api/files/{id}/transcript`	segments	`GET /api/files/{id}/scenes`	scenes
`POST /api/files/{id}/detect-scenes`	single-clip detect	`.../classify-with-ai`	single-clip AI
`GET /api/search`	transcript search	`.../scenes/search`	scene attr + visual search
`POST /api/files/{id}/reveal`	open in Finder	`.../posters`	project poster grid

ANALYSIS, CUT, CHAT, EXPORT

`POST /api/projects/{id}/analyse`	story bible	`GET .../analysis`	fetch bible
`POST /api/projects/{id}/narrative-cut`	AI cut → FCPXML	`GET .../latest.fcpxml`	download
`GET/POST /api/projects/{id}/chat`	project chat	`.../export/transcripts.csv`	CSV
`GET /api/projects/{id}/export`	.nolanproj bundle	`POST /api/projects/import`	import bundle (+relink)

SETTINGS & ADMIN

`GET /api/settings`	read	`PATCH /api/settings`	update
`PATCH /api/settings/api-keys`	set keys (.env)	`POST /api/projects/{id}/relink`	relink footage paths
`GET /api/admin/version`	git HEAD vs origin	`POST /api/admin/update`	git pull + restart

9 Frontend

A single-file SPA — no framework, no build step. `index.html` is a static DOM skeleton; `app.js` holds all logic; the server serves them from `/static`.

9.1 Core state (module-level globals in app.js)

`currentProjectId · activeFileId · allFiles[] · allJobs{} · pollTimer · activeFilter · sortMode · viewMode · scenesCache{} · offlineMode`

9.2 Polling model

While work is active, `startPolling()` hits `/api/projects/{id}/files` + `/api/jobs` every 2.2s and re-renders. Stops when no clip is active AND no background job (`__scene_backfill__` , `__reclassify__`) is running. The **activity console** separately polls `/api/activity` at 1s.

9.3 Clip status model

```
isFullyProcessed = (status∈{done,silent}) && has_thumbnail && has_ai_detect
isPending        = !isFullyProcessed
// Tabs: All · Processed · Pending
```

9.4 Key render functions

FUNCTION	RENDERS
`renderClipGrid()`	main clip list (list/gallery, grouped by status)
`sceneCardHtml()`	scene thumbnails + visual-content tags
`renderSearchResults()`	transcript + scene matches
`updateNowProcessing()`	the bins-panel progress widget
`pollConsole()`	the activity drawer

10 Telegram Bot (`telegram_bot.py`)

Optional. Starts as an asyncio task at boot if `telegram_token` is set. Long-polling (`drop_pending_updates=True`). Lets the editor query footage from their phone.

10.1 Commands

`/start · /projects · /use <id> · /search <q> · /vibe <feel> · /scenes <q> · /stats · /model <name> · /story · /characters · /themes · /summary · /locations · /instructions · /relink`

Plain messages → AI chat over transcripts with tappable "Open in Finder" buttons for cited clips.

10.2 Capture-mode state machine

Two dicts track multi-message flows; `_clear_waiting(chat_id)` is called at the start of every command handler to prevent stuck states:

STATE	MEANING
`_waiting_instructions[chat]`	"add" or "set" — next message becomes a custom instruction
`_waiting_relink[chat]`	next message is a footage folder path to relink
`_pending_cut[chat]`	find→confirm→style→cut conversational flow

10.3 Access control

`telegram_chat_ids` allow-list. First `/start` when the list is empty auto-registers that user (owner onboarding). Per-chat custom instructions are injected into LLM system prompts.

> **LESSON** Several historical bugs came from "stuck capture mode" and a `chat_id` not being threaded into a helper. Always pass `chat_id` explicitly and clear capture state on every command.

11 Narrative Cut → FCPXML (`fcpxml.py`)

Turns a theme + duration into an editable timeline for DaVinci Resolve / Final Cut.

```
theme + duration
 ▼ LLM query-expand → gather candidate clips (transcript + scene pools)
 ▼ select_narrative_clips() → JSON {title, clips:[{filename,in,out}], note}
 ▼ validate clips exist on disk
 ▼ generate_fcpxml() → writes narrative_cuts/<title>_<ts>.fcpxml
 ▼ served / sent via Telegram as a document
```

`_get_tc_info()` reads embedded SMPTE timecode, framerate and audio/video specs per source via `ffprobe` so the XML lines up with the original media in the NLE.

12 Settings & Configuration

12.1 `settings.json` (repo root, gitignored)

KEY	DEFAULT	EFFECT
`offline_mode`	false	Disable all AI (transcription still local)
`scene_detect_mode`	"sampling"	"cuts" = PySceneDetect instead
`scene_broll_seconds`	6	B-roll sample interval
`scene_aroll_seconds`	15	A-roll sample interval
`scene_max_samples`	20	Cap samples per clip
`scene_concurrency`	CPU-1	Parallel scene workers (2–8)
`telegram_token`	—	Bot token (enables bot)
`telegram_chat_ids`	[]	Allow-list
`telegram_default_project_id`	—	Default project for bot
`telegram_model`	"haiku"	Bot LLM
`telegram_instructions`	{}	Per-chat custom rules

12.2 `.env` — API keys

`ANTHROPIC_API_KEY · GROQ_API_KEY · GEMINI_API_KEY`

All optional. With none set, Nolan falls back to the Claude CLI (OAuth) for AI and stays fully functional for transcription/scene detection offline.

13 Deployment & Packaging

13.1 Install

```
git clone https://github.com/mavestone/nolan-studio.git
cd nolan-studio && ./install.sh
```

`install.sh` ensures Homebrew, installs **ffmpeg**, ensures Python 3.11+, builds a venv, `pip install -r requirements.txt`, scaffolds `.env`, then builds `Nolan.app` via `make-app.sh` into /Applications (or chosen dir).

13.2 The app bundle

`make-app.sh` wraps a compiled Swift launcher (`launcher/nolan-launcher`, universal binary). The launcher registers with WindowServer (dock dot, no infinite bounce), writes the repo path to `Resources/nolan-root.txt`, sets the subprocess `currentDirectoryURL` to the repo, spawns Python, waits for `:8765`, opens the browser, and tears down Python on quit (SIGTERM).

13.3 In-app updater

`POST /api/admin/update` does `git pull` then triggers a self-restart (`open -a Nolan` detached + SIGTERM to self). The frontend polls `/api/admin/version` until the server returns, then reloads.

13.4 Startup maintenance (sequential, on every boot)

`_startup_maintenance()` runs once at lifespan start, in order:

1. `purge_orphaned_rows()` — delete files/scenes from deleted projects + VACUUM
2. `cleanup_db_hidden_and_dupes()` — drop ._ sidecars, smart-merge duplicate paths
3. `migrate_error_clips_to_silent()`
4. `backfill_missing_posters()` — wire posters from existing scenes (cheap)
5. `migrate_dialogue_tiers()` — a_roll/b_roll → 3-tier (no video decode)
6. `reclassify_all_scenes()` — guarded: only never-classified scenes

14 Known Gotchas & Lessons

The traps that cost real debugging time. New developers should read this section first.

1 · FFMPEG PATH IN .APP `.app` launches inherit `PATH=/usr/bin:/bin:/usr/sbin:/sbin` — **no Homebrew**. Bare `ffmpeg` / `ffprobe` calls fail silently (ffprobe→0.0 duration, ffmpeg→no thumbnails). Fixed two ways: `main.py` prepends `/opt/homebrew/bin` + `/usr/local/bin` to `os.environ["PATH"]` at startup, and `scene_detector` resolves the binaries to absolute paths. Transcription was unaffected because faster-whisper uses a bundled decoder, which masked the problem.

2 · SQLITE FOREIGN KEYS DEFAULT OFF Deleting a project did **not** cascade — orphaned rows piled up (thousands). `_db()` now sets `PRAGMA foreign_keys=ON` and a startup purge sweeps legacy orphans.

3 · "HAS SCENES" ≠ "PROCESSED" Empty scene rows (failed thumbnail) made `has_scenes()` true so Process skipped clips forever. Gate on `has_scene_thumbnails()` + `has_scene_ai()`.

4 · HEVC FRAME-SKIP IS A NO-OP You cannot cheaply sample "every Nth frame" of HEVC — inter-frame compression forces decoding reference frames anyway. Only keyframe-only decode is genuinely cheap. This is why interval **seeking** (not frame-skipping) is the fast path.

5 · IMPORT THEN RE-SCAN DUPLICATES Importing a `.nolanproj` stores the sender's paths; a local re-scan creates a second row. The cleanup smart-merges: if the missing-path row has transcripts, its path is rewritten to the local one rather than deleted.

6 · FOLDER PICKER PERMISSIONS Use bare `choose folder` osascript (StandardAdditions), **not** `tell application "Finder"` (needs Automation grant). On any picker failure the UI falls back to a manual path text-input.

7 · CACHE-BUSTING Unbundled frontend — bump `app.js?v=NN` in `index.html` after JS/CSS edits or users get stale code.

VERIFYING CHANGES The server logs to `~/Library/Logs/Nolan/server.log` (rotated). Tail it, or watch the in-app Activity console, to see real-time pipeline output. `footage.db` can be inspected directly with the `sqlite3` CLI while the server runs (WAL mode allows concurrent reads).

14.1 Suggested first tasks for a new dev

1. Run `./install.sh`, create a project, scan a small folder, hit Process — watch the Activity console.
2. Read `main.py` top-to-bottom: startup → lifespan → the `/process` handler.
3. Trace one clip through `detect_scenes_sampled()` in `scene_detector.py`.
4. Open `footage.db` in `sqlite3` and look at `scenes` for a processed clip.

End of document · Nolan Developer Handoff · generated from source at github.com/mavestone/nolan-studio